USING WINDOW FUNCTIONS FOR PAGINATION

Window functions offer yet another way to implement pagination in SQL. This is a flexible, and above all, standards-compliant method. However, only SQL Server and the Oracle database can use them for a pipelined top-N query. PostgreSQL does not abort the index scan after fetching enough rows and therefore executes these queries very inefficiently. MySQL does not support window functions at all.

The following example uses the window function ROW_NUMBER for a pagination query:

```
SELECT *
  FROM ( SELECT sales.*
              , ROW_NUMBER() OVER (ORDER BY sale_date DESC
                                          , sale_id   DESC) rn
           FROM sales
       ) tmp
 WHERE rn between 11 and 20
 ORDER BY sale_date DESC, sale_id DESC;
```

The ROW_NUMBER function enumerates the rows according to the sort order defined in the **over** clause. The outer **where** clause uses this enumeration to limit the result to the second page (rows 11 through 20).

The Oracle database recognizes the abort condition and uses the index on SALE_DATE and SALE_ID to produce a pipelined top-N behavior:

```
-------------------------------------------------------------------
|Id | Operation                     | Name    | Rows |  Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT              |         | 1004K| 36877 |
|*1 |  VIEW                         |         | 1004K| 36877 |
|*2 |   WINDOW NOSORT STOPKEY       |         | 1004K| 36877 |
| 3 |    TABLE ACCESS BY INDEX ROWID | SALES   | 1004K| 36877 |
| 4 |     INDEX FULL SCAN DESCENDING | SL_DTID | 1004K|  2955 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
1 - filter("RN">=11 AND "RN"<=20)
2 - filter(ROW_NUMBER() OVER (
           ORDER BY "SALE_DATE" DESC, "SALE_ID" DESC )<=20)
```


EX LIBRIS GHEORGHE GABRIEL SICHIM <GABRIEL.SICHIM@GMAIL.COM>

The `WINDOW NOSORT STOPKEY` operation indicates that there is no sort operation (`NOSORT`) and that the database aborts the execution when reaching the upper threshold (`STOPKEY`). Considering that the aborted operations are executed in a pipelined manner, it means that this query is as efficient as the offset method explained in the previous section.

The strength of window functions is not pagination, however, but analytical calculations. If you have never used window functions before, you should definitely spend a few hours studying the respective documentation.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 8

Modifying Data

So far we have only discussed query performance, but SQL is not only about queries. It supports data manipulation as well. The respective commands—`insert`, `delete`, and `update`—form the so-called "data manipulation language" (DML)—a section of the SQL standard. The performance of these commands is for the most part negatively influenced by indexes.

An index is pure redundancy. It contains only data that is also stored in the table. During write operations, the database must keep those redundancies consistent. Specifically, it means that `insert`, `delete` and `update` not only affect the table but also the indexes that hold a copy of the affected data.

Insert

The number of indexes on a table is the most dominant factor for `insert` performance. The more indexes a table has, the slower the execution becomes. The `insert` statement is the only operation that cannot directly benefit from indexing because it has no `where` clause.

Adding a new row to a table involves several steps. First, the database must find a place to store the row. For a regular heap table—which has no particular row order—the database can take any table block that has enough free space. This is a very simple and quick process, mostly executed in main memory. All the database has to do afterwards is to add the new entry to the respective data block.

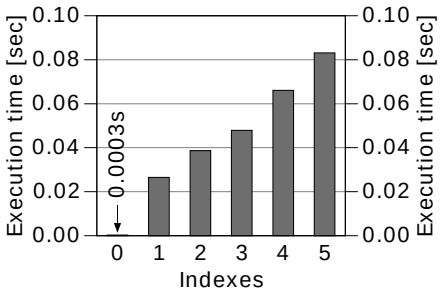
If there are indexes on the table, the database must make sure the new entry is also found via these indexes. For this reason it has to add the new entry to each and every index on that table. The number of indexes is therefore a multiplier for the cost of an **`insert`** statement.

Moreover, adding an entry to an index is much more expensive than inserting one into a heap structure because the database has to keep the index order and tree balance. That means the new entry cannot be written to any block—it belongs to a specific leaf node. Although the database uses the index tree itself to find the correct leaf node, it still has to read a few index blocks for the tree traversal.

Once the correct leaf node has been identified, the database confirms that there is enough free space left in this node. If not, the database splits the leaf node and distributes the entries between the old and a new node. This process also affects the reference in the corresponding branch node as that must be duplicated as well. Needless to say, the branch node can run out of space as well so it might have to be split too. In the worst case, the database has to split all nodes up to the root node. This is the only case in which the tree gains an additional layer and grows in depth.

The index maintenance is, after all, the most expensive part of the **`insert`** operation. That is also visible in Figure 8.1, "Insert Performance by Number of Indexes": the execution time is hardly visible if the table does not have any indexes. Nevertheless, adding a single index is enough to increase the execute time by a factor of a hundred. Each additional index slows the execution down further.

Figure 8.1. Insert Performance by Number of Indexes



Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>



Note

The first index makes the greatest difference.

To optimize **insert** performance, it is very important to keep the number of indexes small.



Tip

Use indexes deliberately and sparingly, and avoid redundant indexes whenever possible. This is also beneficial for **delete** and **update** statements.

Considering **insert** statements only, it would be best to avoid indexes entirely—this yields by far the best **insert** performance. However tables without indexes are rather unrealistic in real world applications. You usually want to retrieve the stored data again so that you need indexes to improve query speed. Even write-only log tables often have a primary key and a respective index.

Nevertheless, the performance without indexes is so good that it can make sense to temporarily drop all indexes while loading large amounts of data—provided the indexes are not needed by any other SQL statements in the meantime. This can unleash a dramatic speed-up which is visible in the chart and is, in fact, a common practice in data warehouses.



Think about it

How would Figure 8.1 change when using an index organized table or clustered index?

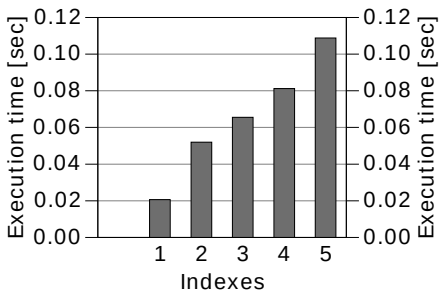
Is there any indirect way an **insert** statement could possibly benefit from indexing? That is, could an additional index make an **insert** statement faster?

Delete

Unlike the **insert** statement, the **delete** statement has a **where** clause that can use all the methods described in Chapter 2, "*The Where Clause*", to benefit directly from indexes. In fact, the **delete** statement works like a **select** that is followed by an extra step to delete the identified rows.

The actual deletion of a row is a similar process to inserting a new one—especially the removal of the references from the indexes and the activities to keep the index trees in balance. The performance chart shown in Figure 8.2 is therefore very similar to the one shown for **insert**.

Figure 8.2. Delete Performance by Number of Indexes



In theory, we would expect the best **delete** performance for a table without any indexes—as it is for **insert**. If there is no index, however, the database must read the full table to find the rows to be deleted. That means deleting the row would be fast but finding would be very slow. This case is therefore not shown in Figure 8.2.

Nevertheless it can make sense to execute a **delete** statement without an index just as it can make sense to execute a **select** statement without an index if it returns a large part of the table.



Tip

Even **delete** and **update** statements have an execution plan.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

A **delete** statement without **where** clause is an obvious example in which the database cannot use an index, although this is a special case that has its own SQL command: **truncate table**. This command has the same effect as **delete** without **where** except that it deletes all rows in one shot. It is very fast but has two important side effects: (1) it does an implicit **commit** (exception: PostgreSQL); (2) it does not execute any triggers.

Side Effects of MVCC

Multiversion concurrency control (MVCC) is a database mechanism that enables non-blocking concurrent data access and a consistent transaction view. The implementations, however, differ from database to database and might even have considerable effects on performance.

The PostgreSQL database, for example, only keeps the version information (=visibility information) on the table level: deleting a row just sets the "deleted" flag in the table block. PostgreSQL's delete performance therefore *does not* depend on the number of indexes on the table. The physical deletion of the table row and the related index maintenance is carried out only during the VACCUM process.

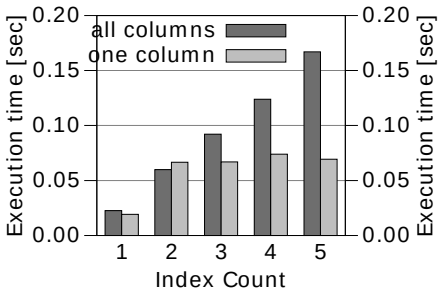
Update

An **update** statement must relocate the changed index entries to maintain the index order. For that, the database must remove the old entry and add the new one at the new location. The response time is basically the same as for the respective **delete** and **insert** statements together.

The **update** performance, just like **insert** and **delete**, also depends on the number of indexes on the table. The only difference is that **update** statements do not necessarily affect all columns because they often modify only a few selected columns. Consequently, an **update** statement does not necessarily affect all indexes on the table but only those that contain updated columns.

Figure 8.3 shows the response time for two **update** statements: one that sets all columns and affects all indexes and then a second one that updates a single column so it affects only one index.

Figure 8.3. Update Performance by Indexes and Column Count



The **update** on all columns shows the same pattern we have already observed in the previous sections: the response time grows with each additional index. The response time of the **update** statement that affects only one index does not increase so much because it leaves most indexes unchanged.

To optimize **update** performance, you must take care to only update those columns that were changed. This is obvious if you write the **update** statement manually. ORM tools, however, might generate **update** statements that set all columns every time. Hibernate, for example, does this when disabling the *dynamic-update mode*. Since version 4.0, this mode is enabled by default.

When using ORM tools, it is a good practice to occasionally enable query logging in a development environment to verify the generated SQL statements. The tip entitled "Enabling SQL Logging" on page 95 has a short overview of how to enable SQL logging in some widely used ORM tools.



Think about it

Can you think of a case where **insert** or **delete** statements do not affect all indexes of a table?

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Appendix A

Execution Plans

Before the database can execute an SQL statement, the optimizer has to create an execution plan for it. The database then executes this plan in a step-by-step manner. In this respect, the optimizer is very similar to a compiler because it translates the source code (SQL statement) into an executable program (execution plan).

The execution plan is the first place to look when searching for the cause of slow statements. The following sections explain how to retrieve and read an execution plan to optimize performance in various databases.

Contents

Oracle Database 166
- Getting an Execution Plan 166
- Operations 167
- Distinguishing Access and Filter-Predicates 170

PostgreSQL 172
- Getting an Execution Plan 172
- Operations 174
- Distinguishing Access and Filter-Predicates 177

SQL Server 180
- Getting an Execution Plan 180
- Operations 182
- Distinguishing Access and Filter-Predicates 185

MySQL 188
- Getting an Execution Plan 188
- Operations 188
- Distinguishing Access and Filter-Predicates 190

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Oracle Database

Most development environments (IDEs) can very easily show an execution plan but use very different ways to format them on the screen. The method described in this section delivers the execution plan as shown throughout the book and only requires the Oracle database in release 9*i*R2 or newer.

Getting an Execution Plan

Viewing an execution plan in the Oracle database involves two steps:

1. **`explain plan for`** — saves the execution plan in the PLAN_TABLE.
2. Format and display the execution plan.

Creating and Saving an Execution Plan

To create an execution plan, you just have to prefix the respective SQL statement with **`explain plan for`**:

```
EXPLAIN PLAN FOR select * from dual;
```

You can execute the **`explain plan for`** command in any development environment or SQL*Plus. It will, however, not show the plan but save it into a table named PLAN_TABLE. Starting with release 10*g*, this table is automatically available as a global temporary table. With previous releases, you have to create it in each schema as needed. Ask your database administrator to create it for you or to provide the **`create table`** statement from the Oracle database installation:

```
$ORACLE_HOME/rdbms/admin/utlxplan.sql
```

You can execute this statement in any schema you like to create the PLAN_TABLE in this schema.



Warning

The **`explain plan for`** command does not necessarily create the same execution plan as though it would when executing the statement.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Showing Execution Plans

The package DBMS_XPLAN was introduced with release 9*i*R2 and can format and display execution plans from the PLAN_TABLE. The following example shows how to display the last execution plan that was explained in the current database session:

```
select * from table(dbms_xplan.display);
```

Once again, if that statement doesn't work out of the box, you should ask your DBA for assistance.

The query will display the execution plan as shown in the book:

```
-------------------------------------------------------------------
| Id | Operation         | Name | Rows | Bytes | Cost (%CPU)|.
-------------------------------------------------------------------
|  0 | SELECT STATEMENT  |      |    1 |     2 |     2   (0)|.
|  1 |  TABLE ACCESS FULL| DUAL |    1 |     2 |     2   (0)|.
-------------------------------------------------------------------
```

Some of the columns shown in this execution plan were removed in the book for a better fit on the page.

Operations

Index and Table Access

INDEX UNIQUE SCAN
: The INDEX UNIQUE SCAN performs the B-tree traversal only. The database uses this operation if a unique constraint ensures that the search criteria will match no more than one entry. See also Chapter 1, "*Anatomy of an Index*".

INDEX RANGE SCAN
: The INDEX RANGE SCAN performs the B-tree traversal *and* follows the leaf node chain to find all matching entries. See also Chapter 1, "*Anatomy of an Index*".

  The so-called index filter predicates often cause performance problems for an INDEX RANGE SCAN. The next section explains how to identify them.

INDEX FULL SCAN
: Reads the entire index—all rows—in index order. Depending on various system statistics, the database might perform this operation if it needs all rows in index order—e.g., because of a corresponding **`order by`** clause. Instead, the optimizer might also use an `INDEX FAST FULL SCAN` and perform an additional sort operation. See Chapter 6, "*Sorting and Grouping*".

INDEX FAST FULL SCAN
: Reads the entire index—all rows—as stored on the disk. This operation is typically performed instead of a full table scan if all required columns are available in the index. Similar to `TABLE ACCESS FULL`, the `INDEX FAST FULL SCAN` can benefit from multi-block read operations. See Chapter 5, "*Clustering Data*".

TABLE ACCESS BY INDEX ROWID
: Retrieves a row from the table using the `ROWID` retrieved from the preceding index lookup. See also Chapter 1, "*Anatomy of an Index*".

TABLE ACCESS FULL
: This is also known as full table scan. Reads the entire table—all rows and columns—as stored on the disk. Although multi-block read operations improve the speed of a full table scan considerably, it is still one of the most expensive operations. Besides high IO rates, a full table scan must inspect all table rows so it can also consume a considerable amount of CPU time. See also "Full Table Scan" on page 13.

Joins

Generally join operations process only two tables at a time. In case a query has more joins, they are executed sequentially: first two tables, then the intermediate result with the next table. In the context of joins, the term "table" could therefore also mean "intermediate result".

NESTED LOOPS JOIN
: Joins two tables by fetching the result from one table and querying the other table for each row from the first. See also "Nested Loops" on page 92.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

HASH JOIN
: The hash join loads the candidate records from one side of the join into a hash table that is then probed for each row from the other side of the join. See also “Hash Join” on page 101.

MERGE JOIN
: The merge join combines two sorted lists like a zipper. Both sides of the join must be presorted. See also “Sort Merge” on page 109.

Sorting and Grouping

SORT ORDER BY
: Sorts the result according to the **`order by`** clause. This operation needs large amounts of memory to materialize the intermediate result (not pipelined). See also “Indexing Order By” on page 130.

SORT ORDER BY STOPKEY
: Sorts a subset of the result according to the **`order by`** clause. Used for top-N queries if pipelined execution is not possible. See also “Querying Top-N Rows” on page 143.

SORT GROUP BY
: Sorts the result set on the **`group by`** columns and aggregates the sorted result in a second step. This operation needs large amounts of memory to materialize the intermediate result set (not pipelined). See also “Indexing Group By” on page 139.

SORT GROUP BY NOSORT
: Aggregates a presorted set according the **`group by`** clause. This operation does not buffer the intermediate result: it is executed in a pipelined manner. See also “Indexing Group By” on page 139.

HASH GROUP BY
: Groups the result using a hash table. This operation needs large amounts of memory to materialize the intermediate result set (not pipelined). The output is not ordered in any meaningful way. See also “Indexing Group By” on page 139.

Top-N Queries

The efficiency of top-N queries depends on the execution mode of the underlying operations. They are very inefficient when aborting non-pipelined operations such as `SORT ORDER BY`.

COUNT STOPKEY
: Aborts the underlying operations when the desired number of rows was fetched. See also the section called "Querying Top-N Rows".

WINDOW NOSORT STOPKEY
: Uses a window function (**`over`** clause) to abort the execution when the desired number of rows was fetched. See also "Using Window Functions for Pagination" on page 156.

Distinguishing Access and Filter-Predicates

The Oracle database uses three different methods to apply **`where`** clauses (predicates):

Access predicate ("access")
: The access predicates express the start and stop conditions of the leaf node traversal.

Index filter predicate ("filter" for index operations)
: Index filter predicates are applied during the leaf node traversal only. They do not contribute to the start and stop conditions and do not narrow the scanned range.

Table level filter predicate ("filter" for table operations)
: Predicates on columns that are not part of the index are evaluated on table level. For that to happen, the database must load the row from the table first.

Execution plans that were created using the DBMS_XPLAN utility (see “Getting an Execution Plan” on page 166), show the index usage in the “Predicate Information” section below the tabular execution plan:

```
-----------------------------------------------------
| Id | Operation         | Name       | Rows  | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |            |     1 | 1445 |
|  1 |  SORT AGGREGATE   |            |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW |  4485 | 1445 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=:A AND "ID2"=:B)
       filter("ID2"=:B)
```

The numbering of the predicate information refers to the “Id” column of the execution plan. There, the database also shows an asterisk to mark operations that have predicate information.

This example, taken from the chapter “Performance and Scalability”, shows an INDEX RANGE SCAN that has access and filter predicates. The Oracle database has the peculiarity of also showing some filter predicate as access predicates—e.g., ID2=:B in the execution plan above.



Important

If a condition shows up as filter predicate, it is a filter predicate—it does not matter if it is also shown as access predicate.

This means that the INDEX RANGE SCAN scans the entire range for the condition "SECTION"=:A and applies the filter "ID2"=:B on each row.

Filter predicates on table level are shown for the respective table access such as TABLE ACCESS BY INDEX ROWID or TABLE ACCESS FULL.

PostgreSQL

The methods described in this section apply to PostgreSQL 8.0 and later.

Getting an Execution Plan

A PostgreSQL execution plan is fetched by putting the **explain** command in front of an SQL statement. There is, however, one important limitation: SQL statements with bind parameters (e.g., `$1`, `$2`, `etc.`) cannot be explained this way—they need to be prepared first:

```
PREPARE stmt(int) AS SELECT $1;
```

Note that PostgreSQL uses "`$n`" for bind parameters. Your database abstraction layer might hide this so you can use question marks as defined by the SQL standard.

The execution of the prepared statement can be explained:

```
EXPLAIN EXECUTE stmt(1);
```

Up till PostgreSQL 9.1, the execution plan was already created with the **prepare** call and could therefore not consider the actual values provided with **execute**. Since PostgreSQL 9.2 the creation of the execution plan is postponed until execution and thus can consider the actual values for the bind parameters.



Note

Statements without bind parameters can be explained directly:

```
EXPLAIN SELECT 1;
```

In this case, the optimizer has always considered the actual values during query planning. If you use PostgreSQL 9.1 or earlier and bind parameters in your program, you should also use **explain** with bind parameters to retrieve the same execution plan.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

The explain plan output is as follows:

```
                QUERY PLAN
-----------------------------------------
 Result  (cost=0.00..0.01 rows=1 width=0)
```

The output has similar information as the Oracle execution plans shown throughout the book: the operation name ("Result"), the related cost, the row count estimate, and the expected row width.

Note that PostgreSQL shows two cost values. The first is the cost for the startup, the second is the total cost for the execution if all rows are retrieved. The Oracle database's execution plan only shows the second value.

The PostgreSQL **explain** command has two options. The VERBOSE option provides additional information like fully qualified table names—VERBOSE is usually not very valuable.

The second explain option is ANALYZE. Although it is widely used, I recommend not getting into the habit of using it automatically because it actually executes the statement. That is mostly harmless for **select** statements but it modifies your data when using it for **insert**, **update** or **delete**. To avoid the risk of accidentally modifying your data, you can enclose it in a transaction and perform a rollback afterwards.

The ANALYZE option executes the statement and records actual timing and row counts. That is valuable in finding the cause of incorrect cardinality estimates (row count estimates):

```
BEGIN;
EXPLAIN ANALYZE EXECUTE stmt(1);

                   QUERY PLAN
-------------------------------------------------
 Result  (cost=0.00..0.01 rows=1 width=0)
         (actual time=0.002..0.002 rows=1 loops=1)
 Total runtime: 0.020 ms

ROLLBACK;
```

Note that the plan is formatted for a better fit on the page. PostgreSQL prints the "actual" values on the same line as the estimated values.



Warning

`explain analyze` executes the explained statement, even if the statement is an **`insert`**, **`update`** or **`delete`**.

The row count is the only value that is shown in both parts—in the estimated and in the actual figures. That allows you to quickly find erroneous cardinality estimates.

Last but not least, prepared statements must be closed again:

```
DEALLOCATE stmt;
```

Operations

Index and Table Access

Seq Scan

The `Seq Scan` operation scans the entire relation (table) as stored on disk (like `TABLE ACCESS FULL`).

Index Scan

The `Index Scan` performs a B-tree traversal, walks through the leaf nodes to find all matching entries, and fetches the corresponding table data. It is like an `INDEX RANGE SCAN` followed by a `TABLE ACCESS BY INDEX ROWID` operation. See also Chapter 1, "*Anatomy of an Index*".

The so-called index filter predicates often cause performance problems for an `Index Scan`. The next section explains how to identify them.

Index Only Scan (since PostgreSQL 9.2)

The `Index Only Scan` performs a B-tree traversal and walks through the leaf nodes to find all matching entries. There is no table access needed because the index has all columns to satisfy the query (exception: MVCC visibility information). See also "Index-Only Scan" on page 116.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Bitmap Index Scan / Bitmap Heap Scan / Recheck Cond
: Tom Lane's post to the PostgreSQL performance mailing list is very clear and concise.

> A plain `Index Scan` fetches one tuple-pointer at a time from the index, and immediately visits that tuple in the table. A bitmap scan fetches all the tuple-pointers from the index in one go, sorts them using an in-memory "bitmap" data structure, and then visits the table tuples in physical tuple-location order.
>
> —Tom Lane[1]

Join Operations

Generally join operations process only two tables at a time. In case a query has more joins, they are executed sequentially: first two tables, then the intermediate result with the next table. In the context of joins, the term "table" could therefore also mean "intermediate result".

Nested Loops
: Joins two tables by fetching the result from one table and querying the other table for each row from the first. See also "Nested Loops" on page 92.

Hash Join / Hash
: The hash join loads the candidate records from one side of the join into a hash table (marked with `Hash` in the plan) which is then probed for each record from the other side of the join. See also "Hash Join" on page 101.

Merge Join
: The (sort) merge join combines two sorted lists like a zipper. Both sides of the join must be presorted. See also "Sort Merge" on page 109.

[1] http://archives.postgresql.org/pgsql-performance/2005-12/msg00623.php

Sorting and Grouping

Sort / Sort Key

Sorts the set on the columns mentioned in `Sort Key`. The `Sort` operation needs large amounts of memory to materialize the intermediate result (not pipelined). See also "Indexing Order By" on page 130.

GroupAggregate

Aggregates a presorted set according to the **`group by`** clause. This operation does not buffer large amounts of data (pipelined). See also "Indexing Group By" on page 139.

HashAggregate

Uses a temporary hash table to group records. The `HashAggregate` operation does not require a presorted data set, instead it uses large amounts of memory to materialize the intermediate result (not pipelined). The output is not ordered in any meaningful way. See also "Indexing Group By" on page 139.

Top-N Queries

Limit

Aborts the underlying operations when the desired number of rows has been fetched. See also "Querying Top-N Rows" on page 143.

The efficiency of the top-N query depends on the execution mode of the underlying operations. It is very inefficient when aborting non-pipelined operations such as `Sort`.

WindowAgg

Indicates the use of window functions. See also "Using Window Functions for Pagination" on page 156.



Caution

PostgreSQL cannot execute pipelined top-N queries when using window functions.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>